Análisis de Algoritmos Avanzados

Anomalía Genética · secuencia ADN

BUSCANDO EL PALÍNDROMO MÁS LARGO DEL GENOMA

Tres implementaciones, un mismo objetivo: encontrar la subsecuencia que se lee igual al derecho y al revés dentro de la secuencia de ADN de un patógeno desconocido. Comparamos su lógica, su complejidad y su desempeño real.

YAEL SEBASTIÁN GONZÁLEZ HERNÁNDEZ A01770992

JOSÉ PEDRO RODRÍGUEZ MUNGUÍA A01769631

MARCO AURELIO TAMEZ ROBLES A01770750

ILEANA TAPIA CASTILLO A01771374

El problema

¿QUÉ ES UN PALÍNDROMO GENÉTICO?

Una subcadena del genoma que se lee igual de izquierda a derecha que de derecha a izquierda. Por ejemplo `ACGGCA` no lo es, pero `ACGCA` sí.

Encontrar el más largo no es trivial: revisar cada centro posible y expandir a mano toma O(n²). Los tres algoritmos de esta presentación resuelven ese mismo reto con estrategias distintas.

4 símbolos: A · C · G · T

longitud desconocida (n)

búsqueda por centro

ATTAAAGGTT

TATACCTTCC

CAGGTAACAA

ACCTTCCA

ACCAACCAAC

TTTCGATCTC

Ruta de la presentación

TRES CAMINOS, MISMO DESTINO

MANACHER	expansión desde el centro · O(n) →
SUFFIX ARRAY	suffix array + LCP + RMQ · O(n log² n) →
PALINDROME SA	variante de suffix array · O(n log² n) →

Cada quien explica su implementación y al final comparamos teoría contra el tiempo real que tardaron sobre la secuencia completa.

Manacher

EXPANDIR SIN VOLVER A COMPARAR

Primero se inserta un separador # entre cada carácter (y en los extremos), para tratar palíndromos de longitud par e impar exactamente igual.

Luego se recorre la cadena una sola vez manteniendo el centro C y el borde derecho R del palíndromo más lejano encontrado hasta ese punto.

Si la posición actual cae dentro de ese borde, su radio mínimo se copia por simetría del espejo — sin comparar carácter por carácter otra vez.



i' es el espejo de i respecto a C — su radio ya lo conocemos

Manacher

LA CLAVE: R NUNCA RETROCEDE

Cada vez que una expansión tiene éxito, el borde R avanza. Nunca vuelve atrás. Eso significa que, sumando todas las expansiones a lo largo de las n posiciones de la cadena, el trabajo total sigue acotado por su tamaño.

Un algoritmo de fuerza bruta prueba cada centro y expande desde cero: $O(n^2)$. Manacher reutiliza lo ya calculado y baja eso a $O(n)$ — lineal.

```
for i in range(1, n-1):
    mirror = 2*C - i
    if R > i:
        P[i] = min(R-i, P[mirror])
    while T[i+1+P[i]] == T[i-1-P[i]]:
        P[i] += 1          # esto es lo único
                           # que cuenta como "trabajo nuevo"
    if i+P[i] > R:
        C, R = i, i+P[i]
```

El while sólo avanza R hacia adelante — por eso el total de pasos está acotado por n.

Suffix Array

VERLO COMO BUSCAR UNA SUBCADENA COMÚN

Se construye `T = S + "#" + reverso(S) + "$"`. Un palíndromo centrado en la posición i de S es, en T, una coincidencia entre el sufijo que empieza en i y su posición espejo del lado invertido.

Esa coincidencia se mide con el arreglo LCP (cuánto comparten dos sufijos consecutivos en el suffix array) y una sparse table que responde "¿mínimo LCP en este rango?" en O(1).



Suffix Array

CONSTRUIR, ORDENAR Y CONSULTAR

SUFFIX ARRAY	rank doubling + sort · O(n log² n)
LCP (KASAI)	recorrido lineal · O(n)
SPARSE TABLE	construir O(n log n) · consultar O(1)

El paso que domina es construir el suffix array: log n rondas de duplicación, cada una con un ordenamiento O(n log n). Total: `O(n log² n)`.

O(n log² n)

tiempo total, dominado por el ordenamiento repetido de sufijos en cada ronda de duplicación

Palindrome SA

MISMA IDEA, OTRA IMPLEMENTACIÓN

Esta versión resuelve el mismo problema con las mismas tres piezas — suffix array, LCP de Kasai y sparse table — pero construye el suffix array ordenando directamente tuplas `(rank[i], rank[i+k])` en cada ronda, en vez de comparar contra un arreglo de claves aparte.

El resultado: complejidad idéntica en la teoría — O(n log² n) — pero una implementación distinta línea por línea. Buen recordatorio de que un mismo algoritmo admite varias formas correctas de escribirse.

```
segundo = rank[k:] + [-1]*k
clave = list(zip(rank, segundo))
sa.sort(key=lambda i: clave[i])
# ordena tuplas (rank[i], rank[i+k])
# directamente, ronda tras ronda
```

Verificado: sobre la secuencia completa, las tres implementaciones encuentran exactamente el mismo palíndromo.

Comparación

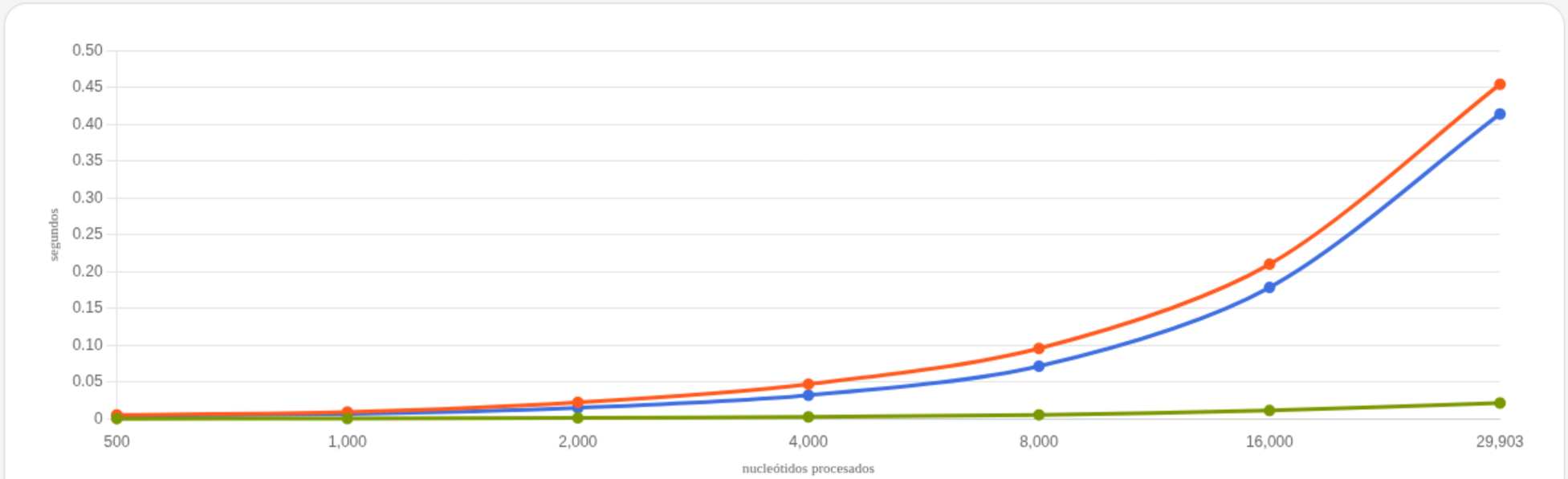
LO QUE DICE LA NOTACIÓN

ALGORITMO	TIEMPO	ESPACIO	IDEA CENTRAL
Manacher	O(n)	O(n)	expansión con simetría de espejo
Suffix Array	O(n log² n)	O(n)	LCS entre S y reverso(S)
Palindrome SA	O(n log² n)	O(n)	misma idea, otro ordenamiento

En teoría Manacher ya se ve favorecido. La pregunta que sigue:
¿se sostiene con datos reales?

La teoría, puesta a prueba

TIEMPO REAL SOBRE LA SECUENCIA DEL PATÓGENO



Manacher Suffix Array Palindrome SA

fragmentos de 500 a 29,903 posiciones (n)

Resultado

MANACHER GANA ESTE RETO

Sobre la secuencia completa (29,903 posiciones), Manacher tardó `0.022s` contra `0.41–0.45s` de las dos versiones de suffix array — casi 20 veces más rápido.

FIRMA GENÉTICA HALLADA: 33 × "A" EN LA POSICIÓN 29,870

Las tres implementaciones coinciden exactamente: es la secuencia espejo que el patógeno oculta en su extremo final — el hallazgo que el equipo buscaba, no un resultado casual.

~20×

más rápido Manacher que cualquiera de los dos suffix arrays, sobre la secuencia completa

Pero el enfoque de suffix array no fue "peor" en general: su LCP + RMQ es un kit reutilizable para muchos otros problemas de cadenas — subcadenas repetidas, comparación entre múltiples secuencias, etc. Manacher está optimizado sólo para este problema.

Conclusiones

TRES FORMAS DE RESOLVER EL MISMO PROBLEMA

ESPECIALIZAR GANA VELOCIDAD	Manacher, hecho a la medida, es O(n) y el más rápido en la práctica
GENERALIZAR GANA ALCANCE	suffix array + LCP resuelve familias enteras de problemas de cadenas
LA IMPLEMENTACIÓN IMPORTA	misma complejidad teórica, distinto código